

PF-3311 — Agentes Virtuales Inteligentes

Entregable 2: Avance de Agente e Investigación

Título: Implementación de un Agente Virtual de Tutoría Socrática Multimodal para el Desarrollo de Lógica de Programación y Resolución de Problemas en Entornos STEM

Nombre del Agente: Sonic — Tutor Socrático Corporizado y Multimodal

Estudiante: Gabriel Isaías Fallas López

Profesor: Alexander Barquero Elizondo

Curso: PF-3311 · Temas Especiales: Agentes Virtuales Inteligentes · Grupo G1

Centro Universitario: San José — Universidad de Costa Rica

Fecha de Entrega: 1 de junio de 2026 · I Semestre 2026

Tabla de Contenidos

- 1. Resumen de Cambios desde el Entregable 1
- 2. Estado Actual del Agente — Prueba de Concepto (PoC)
- 3. Diseño Metodológico del Estudio
- 4. Métricas
- 5. Instrumentos de Recolección de Datos
- 6. Protocolos de Interacción
- 7. Referencias

1. Resumen de Cambios desde el Entregable 1

1.1 Feedback Recibido

El Entregable 1 obtuvo una calificación de **89/100**. El profesor Barquero identificó cuatro áreas de mejora:

Criterio	Nota	Observación del Profesor
Problema y Motivación	20/20	Puntaje perfecto. La tensión entre guiar y no entregar la respuesta está bien definida como eje central.

Criterio	Nota	Observación del Profesor
Revisión de Literatura	19/20	Completa. Se pierde 1 punto por no conectar explícitamente la teoría de andamiaje pedagógico con la lógica técnica del agente.
Preguntas de Investigación	17/20	Se debe sustituir "fluidez pedagógica" por la métrica estándar <i>perceived pedagogical support</i> . Falta condición de control "sin avatar".
Objetivos del Agente	15/15	Puntaje perfecto. Sugerencia: escalar interacciones progresivamente de acuerdo con Vygotsky.
Arquitectura Técnica	8/15	Ready Player Me discontinuó su servicio el 31 de enero de 2026. Se solicita identificar e integrar una alternativa funcional de inmediato y documentar el cambio. Latencia <1.5s es un reto a monitorear.
Redacción, Presentación y Repositorio	10/10	Puntaje perfecto. Sugerencia: incluir instrucciones detalladas de configuración local en el README.

1.2 Cambios Realizados y Justificación

Cambio 1 — Migración del Motor de Avatar y Pivote a Gamificación (corrección crítica + evolución)

Problema original: Ready Player Me cerró su servicio en enero de 2026, eliminando el pilar visual de la arquitectura original (avatares VRM 3D, lip-sync mediante visemas).

Solución adoptada (evolución final): Se migró hacia un **avatar 2D implementado en Kaplay.js**, usando el sprite de Sonic the Hedgehog (16 frames, 2 animaciones: *run / jump*) como representación visual del tutor socrático. El avatar opera mediante 8 estados expresivos reactivos (*idle, thinking, speaking, listening, happy, curious, empathetic, encouraging*) que se mapean a comportamientos de juego en tiempo real (Sonic corre, salta, se congela, activa modo victoria). El sistema incluye dos zonas temáticas que cambian por tarea: **Chemical Plant** (Tarea 1, fondo azul-verde) y **Speed Highway** (Tarea 2, tinte cálido naranja). Un sistema de anillos gamifica el progreso socrático: se gana un anillo cuando la respuesta del usuario se aproxima a la solución, y se pierde cuando el agente detecta frustración. Un mini-juego de transición Kaplay entre tareas refuerza la experiencia lúdica.

Justificación teórica: El Hajji et al. (2025) documentan que avatares 2D estilizados logran el mismo anclaje de atención pedagógica que los 3D con menor costo computacional. La estética *cartoon* de Sonic neutraliza el Valle Inquietante (Mori, 1970) y opera en el territorio de representación claramente artificial pero altamente expresiva. La gamificación con anillos añade un componente de motivación intrínseca (autodeterminación, Deci & Ryan, 2000) que el diseño CSS/SVG original no ofrecía.

Cambio 2 — Actualización de RQ1: "Perceived Pedagogical Support"

Problema: El término "fluidez pedagógica" no corresponde a ningún constructo validado en la literatura HCI ni educativa.

Solución adoptada: Se sustituyó por *perceived pedagogical support*, medido mediante una escala Likert ad hoc de 5 ítems adaptada de Essel et al. (2024). Este constructo está ampliamente documentado en la literatura de Sistemas de Tutoría Inteligente (ITS) y es directamente comparable entre condiciones experimentales.

Cambio 3 — Incorporación de la Condición de Control "Sin Avatar"

Problema: La ausencia de una condición *baseline* hacía imposible atribuir causalmente cualquier efecto observado al embodiment del agente.

Solución adoptada: Se diseñó e implementó la **Condición B**: un chat de texto plano que mantiene el mismo system prompt socrático, el mismo modelo de lenguaje y las mismas tareas, eliminando únicamente el avatar, la voz y los gestos. Esta comparación Condición A (avatar multimodal) vs. Condición B (texto plano) permite aislar el efecto causal del embodiment, respondiendo directamente a la brecha identificada en la literatura (Sonlu et al., 2024).

Cambio 4 — Conexión Explícita entre Andamiaje Pedagógico y Arquitectura Técnica

Problema: La teoría del andamiaje (Vygotsky, ZPD) no estaba vinculada formalmente con la lógica técnica del agente en el Entregable 1.

Solución adoptada: El system prompt socrático (sonic-system.ts) implementa un **protocolo de escalado de pistas en tres niveles** alineado con la Zona de Desarrollo Próximo: (1) preguntas exploratorias de alto nivel, (2) orientación conceptual, y (3) pista estructural específica sin proporcionar el código. El nivel de andamiaje aumenta únicamente si el participante lleva tres o más turnos sin avance. La justificación teórica HCI (Cassell et al., 2000; Bickmore & Picard, 2005; Yee & Bailenson, 2007) fue incorporada en el documento de protocolo de evaluación (docs/03-evaluation-protocol.md).

Cambio 5 — Migración del LLM a Infraestructura Local (Ollama + Gemma 3 12B)

Para garantizar la reproducibilidad de la PoC sin dependencias de API externas, el motor de razonamiento se migró de Google Gemini a **Ollama corriendo Gemma 3 12B localmente** (Q4_K_M, ~7.3 GB, GPU RTX 5070 Ti 16 GB). Esta decisión no afecta la calidad del diálogo socrático y permite ejecutar la PoC de forma autónoma sin claves de API.

Cambio 6 — Migración de TTS y STT a Motores Locales de Alta Calidad

La síntesis de voz migró de Web Speech API a **Piper TTS neuronal** (Docker local, español de alta calidad) con fallback automático a Web Speech API cuando el servidor Piper no está disponible. El reconocimiento de voz migró a **Whisper STT local** via ruta Next.js /api/stt. Ambas decisiones mejoran

la calidad de la experiencia multimodal y mantienen la arquitectura completamente local sin dependencias de nube.

Cambio 7 — Ampliación del README con Instrucciones de Configuración

El README del repositorio fue actualizado con instrucciones detalladas para instalar Ollama, descargar el modelo Gemma 3, configurar el servidor Piper TTS con Docker, configurar variables de entorno y lanzar el servidor de desarrollo Next.js.

2. Estado Actual del Agente — Prueba de Concepto (PoC)

2.1 Descripción General de la PoC

La Prueba de Concepto corresponde a una **aplicación web funcional completa** construida con Next.js 14 (App Router) que implementa los dos brazos del diseño experimental: la **Condición A** — Sonic corporizado con canvas de juego Kaplay 2D, TTS Piper neuronal, STT Whisper, gamificación de anillos, zonas temáticas (Chemical Plant / Speed Highway), mini-juego de transición y SFX/BGM — y la **Condición B** — el mismo tutor socrático presentado como chat de texto plano sin elementos de embodiment. El sistema demuestra la integración real de todos los componentes principales de la arquitectura propuesta.

2.2 Diagrama de Arquitectura



```
|          SERVIDOR NEXT.JS 14 (Node.js)          |
| /api/chat          /api/tts          /api/stt          |
| - sonic-system.ts    - Piper TTS proxy    - Whisper STT proxy |
| - ollamaClient.ts    /api/session          |
| - SSE streaming      - initSession      - logTaskResult    |
| - Avatar state tag    - getSessionSummary - logger.ts      |
+-----+-----+-----+
|                                     | HTTP NDJSON Streaming |
+-----+-----+-----+
|          OLLAMA SERVER (http://localhost:11434)          |
| Modelo: gemma3:12b (Q4_K_M aprox. 7.3 GB)              |
| GPU: NVIDIA RTX 5070 Ti 16 GB VRAM                      |
+-----+-----+-----+
|          PIPER TTS SERVER (Docker, puerto local)          |
| Modelo de voz: espanol alta calidad                    |
| Latencia: menor a 200 ms para respuestas cortas          |
+-----+-----+-----+
```

2.3 Stack Tecnológico Implementado

Capa / Componente	Tecnología Implementada	Justificación
Avatar / Motor de Juego (Condición A)	Kaplay.js 2D + Sprite Sonic the Hedgehog (16 frames, 2 animaciones: run / jump)	Canvas de juego side-scrolling con 8 estados expresivos mapeados a comportamientos de Sonic. Zonas temáticas por tarea (Chemical Plant / Speed Highway). Sistema de anillos como indicador de progreso socrático. Estética cartoon que evita el Valle Inquietante (Mori, 1970).
Motor de Razonamiento (LLM)	Ollama + Gemma 3 12B (Q4_K_M) — inferencia GPU local	Sin API keys externas. Inferencia GPU local (~7.3 GB en RTX 5070 Ti 16 GB VRAM). System prompt socrático (sonic-system.ts) con protocolo ZPD de 3 niveles, detección de frustración y control de avatar via [AVATAR_STATE].
Síntesis de Voz (TTS)	Piper TTS neuronal (Docker local) + fallback Web Speech API	Piper ofrece voz en español de alta calidad con latencia <200 ms. Fallback automático a Web Speech API si Piper no está disponible. Activo únicamente en Condición A.
Reconocimiento de Voz (STT)	Whisper STT local (vía /api/stt en Next.js)	Transcripción de voz del participante. Resultados en tiempo real. Español latinoamericano. Activo únicamente en Condición A.
Gamificación	Sistema de anillos + zonas temáticas + mini-	Anillos gamifican el progreso socrático: se ganan al aproximarse a la solución, se pierden al detectar

Capa / Componente	Tecnología Implementada	Justificación
	juego Kaplay + SFX/BGM	frustración. Mini-juego de transición (~15 s) entre tareas. Efectos de sonido y música de fondo por zona. Exclusivo Condición A.
Mensajes Proactivos	Sistema de pistas automáticas tras 60 s de inactividad	Sonic envía mensajes proactivos específicos por tarea (pool de 4 pistas, selección no repetida) cuando el participante no interactúa durante 60 s. Refuerza el andamiaje ZPD sin intervención del investigador.
Framework / Orquestación	Next.js 14 (App Router) + React + TailwindCSS	SSR para proteger la configuración del servidor, rutas API integradas (/api/chat, /api/tts, /api/stt, /api/session), streaming SSE nativo.
Telemetría / Logs	Módulo logger.ts (en memoria, lado servidor Next.js)	Registra latencia extremo a extremo, turnos, tiempo por tarea, resolución autónoma (sí/no) y anillos acumulados (Condición A) de forma automática.

2.4 Funcionalidades Implementadas vs. Pendientes

Componente	Estado	Detalle
Canvas Kaplay 2D — Sonic (8 estados expresivos)	✓ Implementado	Sprite 16 frames (run/jump), 8 estados mapeados a comportamientos: corre, salta, se congela, modo victoria. Timer bar, ring HUD overlay, tinte de zona.
Zonas temáticas (Chemical Plant / Speed Highway)	✓ Implementado	Zona 1 (fondo azul-verde) — Tarea 1; Zona 2 (tinte naranja cálido) — Tarea 2. Cambia automáticamente al iniciar cada tarea.
Sistema de anillos	✓ Implementado	Ring burst animado al ganar anillo; hurt flash + blink de Sonic al perder. Contador en HUD overlay. Velocidad del fondo escala con anillos acumulados.
Mini-juego de transición Kaplay entre tareas	✓ Implementado	TaskTransitionGame.tsx: Sonic recoge anillos de la tarea anterior, derrota a Motobug, secuencia de outro con barra de carga y fade. ~15 s. Solo Condición A.
TTS Piper neuronal + fallback Web Speech	✓ Implementado	piperTTS.ts: preloadCheck() en mount, Piper vía /api/tts (Docker), fallback automático a Web Speech API. Elimina emojis antes de síntesis. Solo Condición A.

Componente	Estado	Detalle
STT Whisper local	✓ Implementado	whisperSTT.ts vía /api/stt. Detecta disponibilidad antes de habilitar el botón de micrófono. Solo Condición A.
SFX y BGM por zona	✓ Implementado	sfx.ts: efectos de sonido (ring, jump, hurt, victory) y música de fondo por zona. Se detiene en Condición B. Solo Condición A.
Mensajes proactivos por tarea	✓ Implementado	Pool de 4 pistas específicas por task-1 y task-2. Selección aleatoria no repetida. Sonic las envía al chat tras 60 s de inactividad.
LLM socrático — sonic-system.ts (ZPD 3 niveles)	✓ Implementado	Personalidad Sonic energético, restricciones absolutas (sin código directo), detección de frustración, control de avatar via [AVATAR_STATE]. Ollama + Gemma 3 12B.
Streaming de respuesta (SSE)	✓ Implementado	Tokens en tiempo real con cursor animado. Avatar state extraído del stream final y aplicado al canvas Kaplay.
Dos tareas de depuración experimentales	✓ Implementado	Tarea 1: print_numbers() bucle infinito (principiante, 10 min). Tarea 2: find_duplicates() $O(n^3) \rightarrow O(n)$ con set (intermedio, 10 min).
Telemetría log-based automática	✓ Implementado	Latencia extremo a extremo, turnos, tiempo total, resolución autónoma (sí/no), anillos acumulados (Condición A).
Condición A (Sonic multimodal) vs. B (texto plano)	✓ Implementado	Landing page con asignación aleatoria o manual. Condición B: mismo LLM socrático, sin canvas, sin audio, sin anillos.
Cuestionarios digitales integrados en flujo web	◦ Parcial	Los instrumentos existen en /context/Cuestionarios-20260522/. Integración en el flujo es desarrollo futuro.

3. Diseño Metodológico del Estudio

3.1 Tipo de Estudio

Se adoptará un diseño de **evaluación piloto con expertos**. Este tipo de estudio es el apropiado dado que:

- La evaluación con usuarios reales (estudiantes de cursos activos) requiere autorización del Comité Ético Científico (CEC) de la UCR, trámite que excede el alcance temporal del proyecto de curso.

- Los expertos en educación STEM o en sistemas de tutoría inteligente permiten obtener retroalimentación pedagógica y técnica de alta calidad con una muestra pequeña, suficiente para validar la viabilidad y usabilidad del prototipo (Nielsen & Landauer, 1993).
- Los hallazgos del piloto informarán el diseño de un estudio controlado con usuarios reales en fases posteriores del proyecto.

Nota del curso: Para PF-3311 se trabajará con pilotos y/o evaluaciones con expertos, dado que la evaluación con usuarios reales requiere autorización del CEC.

3.2 Diseño Experimental

Se utiliza un **diseño entre grupos (between-subjects)** con dos condiciones:

Condición	Descripción	Componentes activos
Condición A — Experimental (Sonic Multimodal + Multimodal)	Interacción con Sonic mediante canvas Kaplay 2D, TTS Piper neuronal (+ fallback Web Speech), STT Whisper, gamificación de anillos, zonas temáticas y SFX/BGM.	Canvas Kaplay 2D (Sonic sprite) · TTS Piper + fallback Web Speech · STT Whisper · LLM socrático (Ollama Gemma 3 12B) · Anillos + Zonas + SFX
Condición B — Control (Chatbot de Texto)	El mismo tutor socrático con idéntico protocolo ZPD y modelo LLM, presentado como chat de texto plano sin ninguna representación visual ni síntesis de voz.	LLM socrático (idéntico) · Entrada y salida de texto únicamente

Al mantener constantes el modelo LLM, el system prompt socrático, las tareas asignadas y el ambiente de evaluación, cualquier diferencia significativa entre condiciones puede atribuirse causalmente a la **presencia del embodiment multimodal**, no a la calidad de la tutoría.

3.3 Participantes

Perfil de la Muestra

- **¿Quiénes?** Docentes, investigadores o profesionales senior con experiencia en educación STEM, diseño de interfaces conversacionales o sistemas de tutoría inteligente.
- **¿Cuántos?** Entre 4 y 8 participantes (2–4 por condición). Para un estudio piloto exploratorio, esta muestra es estándar en HCI y permite identificar los problemas de usabilidad y pertinencia pedagógica más relevantes.
- **Criterios de inclusión:**

- Experiencia demostrable en docencia o investigación en computación, ingeniería o educación STEM.
- Conocimiento básico de Python suficiente para comprender el contexto de las tareas de depuración.
- Acceso a un navegador web moderno (Chrome 90+, Edge 90+ o Safari 15+).
- **Criterios de exclusión:**
 - Conocimiento previo del objetivo o hipótesis del estudio (para evitar el sesgo de demanda).
 - Haber interactuado previamente con la PoC en modo de prueba o desarrollo.

Plan de Reclutamiento

El reclutamiento se realizará mediante invitación directa dentro de la comunidad académica de la ECCI/UCR y los contactos del profesor del curso PF-3311. Los participantes recibirán una descripción breve y neutral del estudio por correo electrónico (sin revelar la hipótesis) y se coordinarán sesiones individuales de aproximadamente 35–40 minutos en un entorno sin distracciones. ****Estructura temporal:**** Consentimiento + demográfico (~3 min) | PANAS PRE (~1 min) | Tarea 1 de depuración (10 min, Chemical Plant) | Transición lúdica (~0.5 min) | Tarea 2 de optimización (10 min, Speed Highway) | Cuestionarios POST (~5 min) | Entrevista breve (~5 min). ****Duración de interacción pura con tareas: 20 minutos**** (10 + 10 min). La participación es voluntaria y puede interrumpirse en cualquier momento.

4. Métricas

Pregunta de Investigación	Variable / Constructo	Métrica Operacional	Tipo
RQ1 — Naturalidad, presencia social y <i>perceived pedagogical support</i>	Antropomorfismo, animacy, amabilidad, inteligencia percibida, apoyo pedagógico	Puntajes Godspeed (1–5 por subescala); puntaje PPS (1–5); puntaje SUS (0–100)	Ordinal / Continua
RQ2 — Eficacia pedagógica autónoma	Resolución autónoma, turnos de diálogo, tiempo por tarea	% tareas resueltas sin código directo; N.º de turnos hasta resolución; tiempo total (seg) — log-based	Continua / Discreta
RQ3 — Carga cognitiva y estado afectivo	Carga mental, esfuerzo, frustración; afecto positivo y negativo; motivación intrínseca	Raw TLX NASA-TLX (0–100/dimensión); delta PANAS-SF (PA_post – PA_pre; NA_post – NA_pre); puntaje SIMS	Continua (auto-reporte)

Pregunta de Investigación	Variable / Constructo	Métrica Operacional	Tipo
RQ4 — Latencia y continuidad conversacional	Latencia extremo a extremo, tasa de respuestas <1.5 s	Latencia promedio y máxima (ms); % respuestas <1.5 s; tasa de errores de sincronía audio-visual (Cond. A)	Continua (log automático)

Conexión entre métricas y RQs: El Godspeed Questionnaire (Bartneck et al., 2009) es el estándar en HRI/HCI para medir percepciones de agentes virtuales. Las métricas log-based objetivan la eficacia pedagógica eliminando el sesgo de deseabilidad social. La administración del PANAS-SF en dos momentos (pre y post) permite calcular el delta afectivo inducido por la modalidad del agente, controlando el estado emocional basal. La latencia se mide automáticamente en el backend como diferencia entre la recepción del mensaje del usuario y el envío del último chunk del stream.

5. Instrumentos de Recolección de Datos

Instrumento	¿Qué mide?	Base en Literatura	Momento de Administración
Cuestionario Demográfico (ad hoc)	Experiencia en docencia STEM, familiaridad con IA generativa, nivel de Python	Estándar en diseños HCI para controlar variables de experiencia previa	Inicio de sesión — antes de cualquier exposición al agente
PANAS-SF (10 ítems, Likert 1–5)	Estado afectivo: afecto positivo (PA) y afecto negativo (NA)	Thompson (2007) — validación internacional de la forma breve	<i>PRE-interacción</i> (línea base) y <i>POST-interacción</i> (delta afectivo)
Godspeed Questionnaire (5 subescalas semánticas diferenciales)	Antropomorfismo, animacy, amabilidad, inteligencia percibida, seguridad percibida del agente	Bartneck et al. (2009) — estándar de facto en HRI/HCI para percepciones de agentes virtuales	Post-sesión completa (tras ambas tareas)
SUS — System Usability Scale (10 ítems, Likert 1–5)	Usabilidad percibida del sistema (0–100). Puntaje ≥68: aceptable; ≥80: buena; ≥90: excelente	Brooke (1996) — instrumento de usabilidad más utilizado en investigación HCI	Cierre de la sesión, antes de la entrevista cualitativa

Instrumento	¿Qué mide?	Base en Literatura	Momento de Administración
NASA-TLX Raw TLX (6 dimensiones)	Carga de trabajo percibida: demanda mental, temporal, física, rendimiento, esfuerzo, frustración	Hart & Staveland (1988) — instrumento de carga cognitiva más utilizado en Computing Education	Post-interacción completa
SIMS — Situational Intrinsic Motivation Scale (16 ítems, Likert 1–7)	Motivación intrínseca situacional: interés, disfrute, competencia percibida, esfuerzo	Guay, Vallerand & Blanchard (2000) — sensible a motivación situacional, no a disposiciones crónicas	Post-interacción, inmediatamente después de completar o agotar el tiempo de las tareas
Escala de Perceived Pedagogical Support (5 ítems, Likert 1–5 · ad hoc)	Utilidad de las pistas socráticas, sensación de guía, percepción de que el agente comprende el estado cognitivo	Adaptada de Essel et al. (2024) — reemplaza "fluidez pedagógica" del E1	Post-tarea 1 y post-tarea 2
Métricas Log-Based Automáticas	Latencia extremo a extremo (ms), N.º de turnos, tiempo por tarea (seg), resolución autónoma (sí/no)	Siemens & Long (2011) — Learning Analytics para medir eficacia pedagógica objetivamente	Continuo durante toda la sesión, registrado automáticamente en backend
Entrevista Semi-Estructurada (~5 min)	Percepciones cualitativas: utilidad de las pistas socráticas, comparación con IA generativa tradicional, sugerencias de mejora	Preece, Rogers & Sharp (2015) — técnica cualitativa estándar en HCI	Al finalizar todos los cuestionarios post-interacción

5.1 Orden de Administración

- 1. Cuestionario demográfico + PANAS-SF *PRE*
- 2. Sesión experimental — Tarea 1 → Perceived Pedagogical Support (post-tarea)
- 3. Sesión experimental — Tarea 2 → Perceived Pedagogical Support (post-tarea)
- 4. PANAS-SF *POST*
- 5. Godspeed Questionnaire
- 6. SUS — System Usability Scale

- 7. NASA-TLX
- 8. SIMS
- 9. Entrevista semi-estructurada

6. Protocolos de Interacción

Se presentan dos escenarios de interacción detallados, diseñados para ser reproducibles y redactados de forma neutral, sin mencionar los términos "avatar", "embodiment" ni "presencia social", para evitar el sesgo de expectativa del participante (Efecto Proteus, Yee & Bailenson, 2007).

Escenario 1 — Depuración de Bucle Infinito (Principiante)

Contexto presentado al participante:

"Usted está trabajando con un estudiante de primer año de ingeniería. El estudiante le muestra el siguiente programa en Python que, según indica, debería imprimir los números del 1 al 5, pero nunca termina. Su tarea es interactuar con el sistema de apoyo disponible para identificar el error lógico en el código y ser capaz de explicarlo con sus propias palabras. No modifique el código; solo identifique el problema."

Código presentado al participante:

```
def print_numbers():  
    counter = 1  
    while counter <= 5:  
        print(f"Número: {counter}")  
        # ¿Por qué nunca termina?  
  
print_numbers()
```

Tareas específicas a realizar:

1. Observar el código durante 1–2 minutos sin interactuar con el sistema.
2. Iniciar una conversación con el agente describiendo el comportamiento observado.
3. Responder a las preguntas del agente y continuar el diálogo hasta identificar el error lógico.
4. Al identificar el problema, presionar el botón "Resolví el problema" y describir verbalmente la solución.

Resultado esperado de la interacción:

El participante identifica que la variable `counter` nunca se incrementa dentro del bucle `while`, por lo que la condición `counter <= 5` es siempre verdadera. La corrección consiste en añadir

counter += 1 al final del cuerpo del bucle. El agente no menciona directamente el error; guía mediante preguntas sobre la condición de salida y el estado de la variable de control.

Ejemplo de flujo de diálogo esperado:

Turno	Participante	Sonic (ejemplo de respuesta socrática)
1	"El programa no termina nunca."	"Anomalía detectada en la ruta de ejecución. Analicemos la condición de salida: ¿qué valor debe alcanzar counter para que el bucle se detenga?"
2	"counter debe ser mayor a 5."	"Correcto. Ahora dime: ¿dónde exactamente, dentro del cuerpo del bucle, se actualiza el valor de counter?"
3	"No veo que se actualice en ningún lado..."	"Estado del sistema: análisis correcto. Si counter nunca cambia, ¿qué implicación tiene eso para la condición de salida en cada iteración?"
4	"La condición siempre será verdadera. ¡Ya entendí!"	"Excelente deducción autónoma. Has localizado el error de arquitectura. ¿Cómo reestructurarías el bucle para que avance correctamente?"

Duración estimada: 5 minutos

Criterio de éxito: El participante identifica verbalmente que counter no se incrementa, sin que el agente haya revelado directamente el error.

Escenario 2 — Análisis de Complejidad y Optimización Algorítmica (Intermedio)

Contexto presentado al participante:

"Usted está revisando el código de un estudiante de segundo año. El programa funciona correctamente —encuentra los números duplicados en una lista—, pero se vuelve extremadamente lento cuando la lista tiene miles de elementos. El estudiante no entiende por qué. Interactúe con el sistema de apoyo para identificar por qué el algoritmo es ineficiente y qué estructura de datos podría mejorar el rendimiento. No proponga código directamente; solo identifique el problema conceptual."

Código presentado al participante:

```
def find_duplicates(numbers):  
    """Encuentra números duplicados en una lista."""  
    duplicates = []
```

```
for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] == numbers[j]:
            if numbers[i] not in duplicates:
                duplicates.append(numbers[i])
    return duplicates

# Prueba con lista grande
import random
big_list = [random.randint(1, 1000) for _ in range(10000)]
result = find_duplicates(big_list)
print(f"Duplicados encontrados: {len(result)}")
```

Tareas específicas a realizar:

- 1. Analizar la estructura del algoritmo: identificar cuántos bucles existen y qué operaciones realiza cada uno.
- 2. Interactuar con el agente para razonar sobre el concepto de complejidad temporal.
- 3. Identificar cuántas operaciones realiza el algoritmo para una lista de N elementos.
- 4. Proponer, en palabras, qué estructura de datos reduciría la complejidad y explicar por qué.

Resultado esperado de la interacción:

El participante identifica que el algoritmo tiene complejidad $O(n^3)$ en el peor caso: dos bucles anidados $O(n^2)$ más la búsqueda lineal `not in duplicates` $O(n)$. La solución conceptual es utilizar un `set` (conjunto) para rastrear los elementos vistos, reduciendo la complejidad total a $O(n)$. El agente guía mediante preguntas sobre qué hace cada bucle, cuántas veces se ejecuta y qué garantiza un conjunto frente a una lista en cuanto a tiempo de búsqueda.

Ejemplo de flujo de diálogo esperado:

Turno	Participante	Sonic (ejemplo de respuesta socrática)
1	"El código es lento por los dos bucles."	"Diagnóstico inicial prometedor. Si el bucle externo itera N veces y el interno itera aproximadamente N veces también, ¿cuántas comparaciones totales ocurren para N = 10 000?"
2	"Serían unos 50 millones de comparaciones."	"Latencia crítica identificada. Ahora observa la tercera operación dentro del bucle: <code>not in duplicates</code> . ¿Qué estructura es <code>duplicates</code> y cuánto tarda en verificar si un elemento ya existe?"
3	"Es una lista... buscar en ella es $O(n)$."	"Estado del sistema: análisis correcto. Entonces tienes $O(n) \times O(n) \times O(n)$. ¿Qué estructura de datos garantiza búsqueda en tiempo constante $O(1)$, independientemente del tamaño?"

Turno	Participante	Sonic (ejemplo de respuesta socrática)
4	"Un set. Con un set la búsqueda es instantánea."	"Arquitectura optimizada identificada de forma autónoma. ¿Cómo describirías el nuevo algoritmo usando dos conjuntos: uno para elementos vistos y otro para duplicados?"

Duración estimada: 8–10 minutos (tiempo máximo: 10 minutos).

Criterio de éxito: El participante identifica los tres niveles de complejidad y propone el uso de un conjunto (*set*), sin que el agente haya mencionado directamente ninguna de estas respuestas.

6.1 Consideraciones Generales del Protocolo

- **Instrucciones neutras:** Ninguna instrucción entregada al participante menciona "avatar", "embodiment" ni "presencia social", para evitar el sesgo de expectativa.
- **Asignación ciega a condición:** El investigador carga la condición (A o B) sin revelar que existe una condición alternativa ni cuál es el objetivo de la comparación.
- **Tiempo máximo por tarea:** 10 minutos. Si el participante no identifica el error en ese tiempo, la tarea se registra como no resuelta y se avanza a la siguiente.
- **Protocolo de des-escalada de frustración:** Si el participante muestra signos de frustración (más de 3 turnos sin avance), Sonic escala automáticamente el andamiaje al siguiente nivel sin proporcionar el código de corrección.
- **Consentimiento informado:** Todos los participantes firman un consentimiento informado antes de iniciar la sesión (código PF-3311, investigador: Gabriel Fallas López, unidad: ECCI/UCR). Los datos se vinculan únicamente a un ID alfanumérico anónimo.

Referencias

1. Bartneck, C., Kulić, D., Croft, E., & Zoghbi, S. (2009). Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots. *International Journal of Social Robotics*, 1(1), 71–81.
2. Bickmore, T. W., & Picard, R. W. (2005). Establishing and maintaining long-term human–computer relationships. *ACM Transactions on Computer-Human Interaction*, 12(2), 293–327.
3. Brooke, J. (1996). SUS: A "quick and dirty" usability scale. En P. W. Jordan et al. (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
4. Cassell, J., Sullivan, J., Prevost, S., & Churchill, E. (Eds.). (2000). *Embodied conversational agents*. MIT Press.

5. El Hajji, M., et al. (2025). An architecture for intelligent tutoring in virtual reality: Integrating LLMs and multimodal interaction for immersive learning. *Information*, 16(7), 556.
6. Essel, H. B., Vlachopoulos, D., Essuman, A. B., & Amankwa, J. O. (2024). AI-mediated questioning and engagement. *MDPI Sustainability*, 17(21), 9508.
7. Guay, F., Vallerand, R. J., & Blanchard, C. (2000). On the assessment of situational intrinsic and extrinsic motivation: The Situational Motivation Scale (SIMS). *Motivation and Emotion*, 24(3), 175–213.
8. Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. En P. A. Hancock & N. Meshkati (Eds.), *Human mental workload* (pp. 139–183). North-Holland.
9. Mori, M. (1970). The uncanny valley. *Energy*, 7(4), 33–35.
10. Nielsen, J., & Landauer, T. K. (1993). A mathematical model of the finding of usability problems. En *INTERACT '93 and CHI '93 Conference Proceedings* (pp. 206–213). ACM.
11. Preece, J., Rogers, Y., & Sharp, H. (2015). *Interaction design: Beyond human-computer interaction* (4.^a ed.). Wiley.
12. Siemens, G., & Long, P. (2011). Penetrating the fog: Analytics in learning and education. *Educause Review*, 46(5), 30–32.
13. Sonlu, S., et al. (2024). A scoping review of embodied conversational agents in education: Trends and innovations from 2014 to 2024. *ResearchGate*.
14. Thompson, E. R. (2007). Development and validation of an internationally reliable short-form of the Positive and Negative Affect Schedule (PANAS). *Journal of Cross-Cultural Psychology*, 38(2), 227–242.
15. Yee, N., & Bailenson, J. (2007). The Proteus effect: The effect of transformed self-representation on behavior. *Human Communication Research*, 33(3), 271–290.